Agent-to-Agent (A2A) Framework

A three-layer protocol stack for opaque, interoperable agent collaboration

Open standard · Google · v1.0.0 · 50+ enterprise partners · HTTP + JSON-RPC 2.0 + SSE

LAYER 1

Canonical Data Model

Protocol-agnostic objects defined in `spec/a2a.proto` — the single normative source.

- **AgentCard** — capabilities, skills, auth
- **Task** — stateful unit of work
- **Message** — role: user / agent
- **Part** — text · file · data
- **Artifact** — structured outputs
- **Extension** — strongly-typed metadata

LAYER 2

Abstract Operations

Eleven binding-independent operations all conformant servers MUST support.

- **SendMessage** — blocking · returns Task | Message
- **SendStreamingMessage** — SSE · realtime updates
- **SubscribeToTask** — resume / multi-client
- **GetTask · ListTasks** — poll · cursor pagination
- **CancelTask** — idempotent
- **PushNotificationConfig** — Create / Get / List / Delete

LAYER 3

Protocol Bindings

Concrete wire formats — abstract ops mapped to method names, endpoints, transports.

- **JSON-RPC 2.0** — over HTTP(S)
- **gRPC** — protobuf streaming
- **HTTP + JSON** — REST endpoints
- **Custom** — if binding contract met

GUIDING PRINCIPLES

Why A2A looks the way it does

01 Opaque execution
Agents collaborate on declared capabilities — never sharing internal state, memory, or tools.

02 Async-first
Designed for long-running tasks — hours, days, or weeks — with human-in-the-loop pauses.

03 Modality agnostic
Text, files, structured data, audio/video — composed from typed `Parts`.

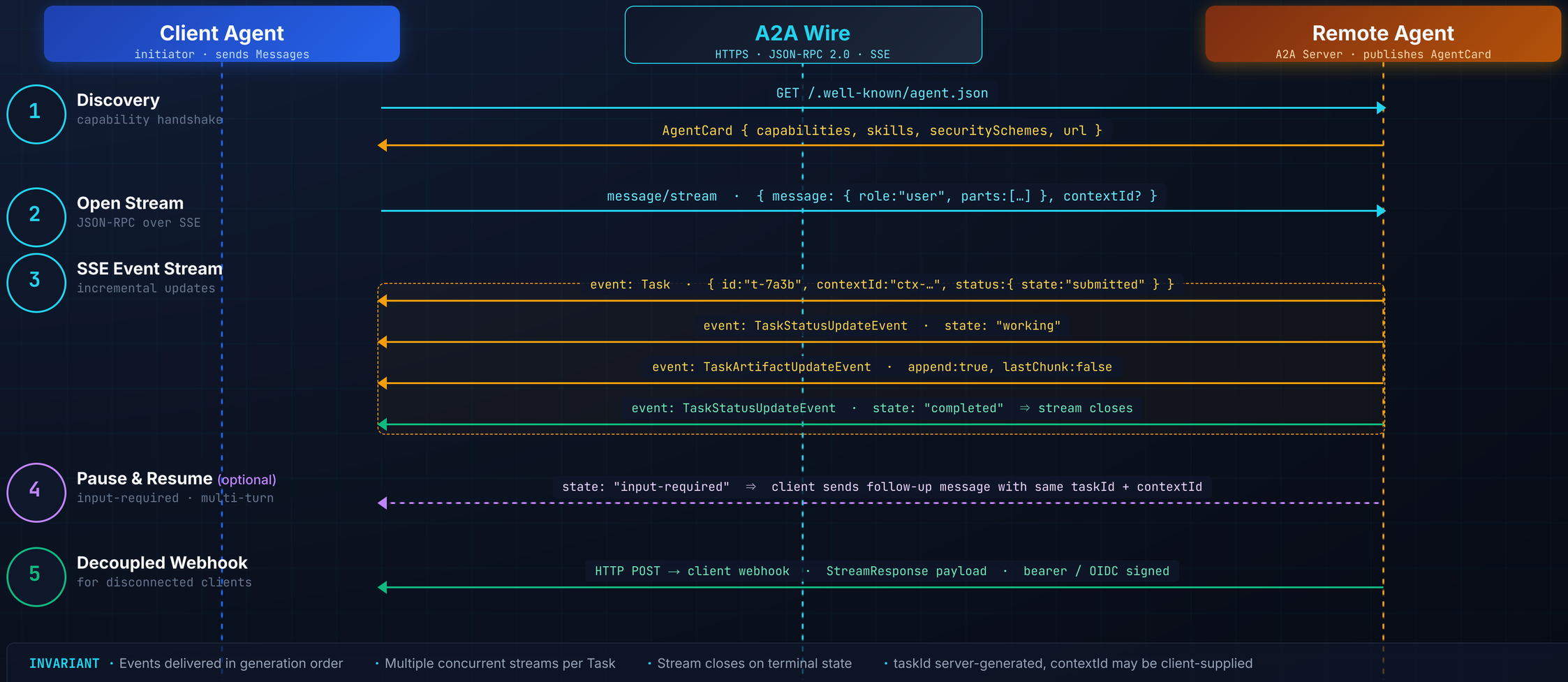
04 Reuse web standards
HTTP, JSON-RPC 2.0, SSE, webhooks — no new transports invented.

05 Horizontal interop
A2A is agent ↔ agent. MCP is agent ↔ tool. They compose — they do not replace each other.

Source · A2A Protocol Specification v1.0.0 (a2aproject/A2A) · spec/a2a.proto is the single normative source

How two agents actually wire up at runtime

Discover · Negotiate · Stream · Resolve — the five-phase protocol flow over JSON-RPC + SSE



INVARIANT · Events delivered in generation order · Multiple concurrent streams per Task · Stream closes on terminal state · taskId server-generated, contextId may be client-supplied

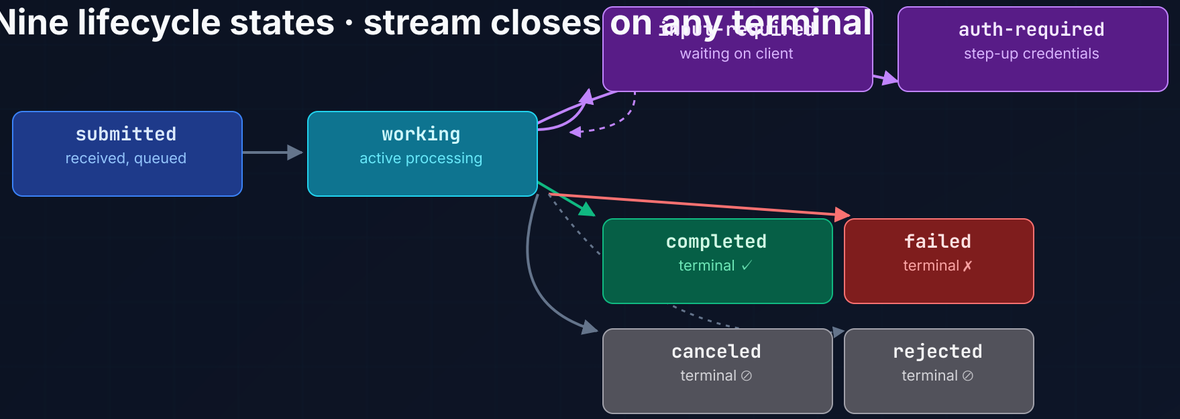
Source · A2A Spec §3.1 Operations · §3.5 Update Delivery Mechanisms · §4.2 Task Events

The stateful machinery behind every A2A task

Lifecycle states · three update channels · transport mapping · enterprise security baseline

TASK STATE MACHINE

Nine lifecycle states · stream closes on any terminal



initial · active · interrupted (multi-turn) · terminal · success · terminal · error · terminal · stopped

`unknown` may briefly appear during reconnect / partial state recovery — agents MUST treat it as non-terminal.

THREE UPDATE CHANNELS

Pick by client topology, not by preference

POLLING

GetTask

Always available · highest latency · best for firewalled / batch clients.

STREAMING

SSE

Real-time · persistent connection · interactive UIs, live dashboards.

PUSH

Webhook

Server-to-server · disconnected clients · long-running orchestration.

METHOD MAPPING · JSON-RPC BINDING

- `message/send` → Send Message
- `message/stream` → Send Streaming Message
- `tasks/get` → Get Task
- `tasks/list` → List Tasks
- `tasks/cancel` → Cancel Task
- `tasks/resubscribe` → Subscribe to Task
- `tasks/pushNotificationConfig/*` → Webhook CRUD
- `agent/getCard` → Get Extended Agent Card

ENTERPRISE GUARANTEES

Security baked into the wire

AUTH

OpenAPI-aligned schemes · OAuth2 / OIDC · JWT · API keys · mTLS

AUTHZ

Per-client task scoping · existence MUST NOT leak to unauthorized callers

TRANSPORT

HTTPS only · TLS 1.2+ · webhook payloads signed

OPAQUE BOUNDARY

No tool / memory / weight sharing — only Tasks, Messages, Artifacts cross the wire

Source · A2A Spec v1.0.0 §3 Operations · §4 Data Model · §9 JSON-RPC Binding · §13 Security